

课程笔记

CS146S Week 1: Introduction to Coding LLMs and AI Development

基于 Mihail Eric 授课内容整理

2025 年 10 月

课程: Stanford CS146S

学期: Fall 2025

讲次: 2 lectures (Mon + Fri)

网站: <https://themodernsoftware.dev>

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1	课程导论与 2025 年软件开发现状	2
1.1	为什么需要这门课	2
1.2	课程核心理念	2
1.3	课程安排概览	2
1.4	本章小结	2
2	LLM 工作原理速览	3
2.1	基本架构	3
2.2	训练过程：三阶段范式	3
2.3	模型规模参考	4
2.4	LLM 编程的优势与局限	4
2.5	本章小结	4
3	LLM Prompting 技术体系	4
3.1	Prompting 基础	4
3.2	核心 Prompting 技术	5
3.2.1	Zero-shot Prompting	5
3.2.2	K-shot Prompting (In-Context Learning)	5
3.2.3	Chain-of-Thought (CoT) Prompting	5
3.2.4	Self-Consistency Prompting	5
3.3	高级技术	5
3.3.1	Tool Use	5
3.3.2	Retrieval Augmented Generation (RAG)	5
3.3.3	Reflexion	5
3.4	Prompt 结构：System / User / Assistant	6
3.5	Prompting 最佳实践	6
3.6	本章小结	6
4	总结与延伸	6
4.1	关键点	6
4.2	拓展阅读	7

1 课程导论与 2025 年软件开发现状

1.1 为什么需要这门课

2025 年，软件开发正经历一场深刻的范式转变。以 Windsurf、Cursor、Claude Code 等为代表的 AI 编程工具正在以前所未有的速度改变开发者的工作方式。Mihail Eric 在课程开篇指出了两个关键信号：

核心观点：AI 不会取代开发者

“You won’t be replaced by AI. You’ll be replaced by a competent engineer who knows how to use AI.”

AI 本身不会取代软件工程师，但善于使用 AI 的工程师将取代不善于使用 AI 的工程师。这门课的目标是让学生成为前者。

好消息是：软件开发拥有历史上最高的生产力潜力。借助 AI 编程工具，工程师可以以前所未有的速度学习新技术栈和工具。

这不是 “Vibe Coding” 课

课程明确强调：这不是一门教你用 AI 随意生成代码的课程。重点在于理解 AI 编程工具的原理、掌握高质量的 human-agent 协作模式，而非盲目依赖 AI。

1.2 课程核心理念

课程围绕以下关键理念展开：

1. **Human-Agent Engineering**：专注于 AI 尚未替代的技能——业务理解、技术架构决策、成为 Tech Lead
2. **LLM 的能力取决于你**：好的 context 产生好的代码；如果你自己都无法理解你的代码库，LLM 也不能
3. **大量阅读和审查代码**：学会辨别好代码与坏代码，培养“品味” (taste)
4. **激进地实验**：目前没有成熟的软件开发范式，每个人都在摸索中

1.3 课程安排概览

10 周课程涵盖 AI 编程工具链的各个层面：从 LLM 基础、编程 Agent 构建、AI IDE、终端工具、测试与安全、代码审查、UI 自动化构建、DevOps，到 AI 软件工程的未来展望。每周邀请行业顶尖创始人做客座讲座。

1.4 本章小结

本节确立了课程的基调：AI 正在改变软件开发，但核心竞争力仍然在于工程师自身的理解力、判断力和对工具的熟练运用。关键是学会如何与 AI Agent 高效协作，而非被 AI 所取代。

2 LLM 工作原理速览

2.1 基本架构

LLM (Large Language Model) 是一种自回归 (autoregressive) 的 next-token prediction 模型。其基本工作流程为：

1. **Tokenization**: 使用固定词汇表将输入文本分割为 token
2. **Embedding**: 将 token 转换为固定维度的数值向量 (通常 1K-3K 维)
3. **Transformer layers**: 经过 12-96+ 层 Transformer (使用 self-attention 机制, Vaswani et al. 2017)
4. **输出概率分布**: 在词汇表上生成下一个 token 的概率分布

Transformer 架构的核心

Self-attention 机制允许模型在生成每个 token 时“看到”输入序列中的所有其他 token, 从而捕获长距离依赖关系。这是 LLM 能够理解代码上下文的关键技术基础。

2.2 训练过程：三阶段范式

现代 LLM 的训练通常遵循三阶段范式：

Stage 1: Self-supervised Pretraining

- 在大量公开数据上进行自监督预训练
- 数据源包括 Common Crawl、Wikipedia、StackExchange、公开 GitHub 仓库等
- 数据规模：数千亿到万亿 + token (语言和代码)
- 目标：学习语言的基本结构和世界知识

Stage 2: Supervised Fine-tuning (SFT)

- 教模型遵循指令
- 使用高质量的 prompt-response 对进行训练
- 数据规模：数万到数十万对
- 例：“Write a for loop” → “ok here’s a for loop...”

Stage 3: Preference Tuning (RLHF/DPO)

- 将模型输出与人类偏好对齐 (helpfulness、correctness、readability)
- 收集同一 prompt 的多个输出，训练 reward model 预测人类偏好
- 数据规模：数万到数十万个人类标注的比较对

Reasoning Models 的演进

在三阶段基础上，reasoning models 进一步扩展了训练：加入 chain-of-thought reasoning traces、工具使用整合、对推理步骤的人类偏好收集，以及通过强化学习 (RL) 学习如何评估推理轨迹和回溯 (backtrack)。

2.3 模型规模参考

- GPT-3 / Claude 3.5 Sonnet: ~175B parameters
- LLaMA 3.1: 405B parameters
- GPT-4: ~1.8T parameters (传闻)

2.4 LLM 编程的优势与局限

优势:

- Expert-level 代码补全
- 代码理解与解释
- 代码修复 (bug fixing)

局限:

- **Hallucination**: 生成不存在或过时的 API (可通过 robust context engineering 缓解)
- **Context window 限制**: ~100–200K tokens, 但并非所有 token 都同等有效
- **延迟**: 每次请求数秒到数分钟 (需要据此规划任务委派)
- **成本**: 最佳模型的输入 token \$1–3/M, 输出 token \$10+/M

Context Window 的质量问题

虽然现代 LLM 宣称支持 100K–200K token 的 context window, 但研究表明模型对窗口中间部分的信息关注度显著下降 (“lost in the middle” 现象)。不能简单地把所有内容塞入 context 就期望好结果。

2.5 本章小结

LLM 本质上是 next-token prediction 模型, 通过三阶段训练 (预训练 → SFT → 偏好对齐) 获得强大的代码生成能力。理解其工作原理和局限性是有效使用 AI 编程工具的前提。

3 LLM Prompting 技术体系

3.1 Prompting 基础

Prompt 是与 LLM 交互的“通用语言” (lingua franca), 也是有效“编程” LLM 的核心手段。正如搜索引擎改变了人们获取信息的方式, prompting 正在改变开发者与 AI 工具的交互方式。

Prompting 的双重性质

Prompting 既是科学也是艺术。LLM 的黑盒性质意味着存在一定的“魔法”成分, 但同时也有经过实证验证的技术可以稳定提升 LLM 表现。

3.2 核心 Prompting 技术

3.2.1 Zero-shot Prompting

最基本的方式：直接要求 LLM 执行任务，不提供任何示例或支持信息。

3.2.2 K-shot Prompting (In-Context Learning)

提供 k 个示例（通常 $k = 1, 3, 5$ ），让 LLM 通过示例学习任务模式。适用于推理步骤较少的任务。

K-shot 的关键价值

K-shot prompting 的核心在于让 LLM 学习你的**特定约定**——比如代码库的命名规范、代码风格等。通过几个精心选择的示例，可以显著提升输出与项目风格的一致性。

3.2.3 Chain-of-Thought (CoT) Prompting

要求 LLM 展示推理步骤，适用于包含多个逻辑步骤的任务（编程、数学等）。两种形式：

- **Multi-shot CoT**：在示例中提供完整的推理过程
- **Zero-shot CoT**：在 prompt 末尾加上 “Think step by step” 等指令

3.2.4 Self-Consistency Prompting

对同一 prompt 采样多次（通常结合 CoT），聚合最常见的结果。通过模型集成（ensembling）的方式减少 hallucination 和错误答案。

3.3 高级技术

3.3.1 Tool Use

允许 LLM 调用外部系统（如测试框架、API 等），是减少 hallucination 和提升 LLM 自主性的最重要技术之一。

3.3.2 Retrieval Augmented Generation (RAG)

为 LLM 注入上下文数据，保持信息的时效性（无需重训模型）。优势包括：更快的迭代、免费获得可解释性和引用来源、减少 hallucination。

3.3.3 Reflexion

让 LLM 反思自己的输出。通过将环境反馈（如测试失败信息）重新注入 context，实现多轮迭代改进。典型模式：

1. 观察 (Observe)：执行动作后观察结果
2. 反思 (Reflect)：分析错误原因
3. 扩展 Prompt (Extend)：基于反思结果改进下一轮的输入

3.4 Prompt 结构: System / User / Assistant

- **System Prompt**: 定义 LLM 的行为、规则和输出风格, 通常用户不可见
- **User Prompt**: 用户的实际请求或指令
- **Assistant Prompt**: LLM 的响应

System Prompt 的威力

System prompt 是定义 LLM “人格” 和行为约束的关键。通过 role prompting (角色设定), 可以显著改变输出质量。例如, 设定 “你是一个 senior software developer 级别的编程助手” 会比默认角色产生更专业的代码输出。

3.5 Prompting 最佳实践

1. **清晰表达**: 如果一个没有上下文的人读了你的 prompt 会感到困惑, LLM 也会
2. **使用结构化格式**: 用 XML 标签 (如 `<log>`、`<error>`、`<code_snippet>`) 组织 prompt 内容
3. **明确约束**: 指定语言、技术栈、库、限制条件
4. **任务分解**: 将复杂任务拆分为更小的子任务
5. **积极使用 Role Prompting**: 通过角色设定强化 system prompt 的效果
6. **利用 Anthropic 的 Prompt Improver**: <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/prompt-improver>

3.6 本章小结

Prompting 是与 LLM 高效交互的基础技能。从 zero-shot 到 CoT、从 tool use 到 RAG 和 reflexion, 不同技术适用于不同场景。核心原则是: 提供清晰的结构化上下文, 明确表达期望, 并利用多轮反馈迭代优化结果。

4 总结与延伸

Week 1 建立了整门课程的认知框架: AI 正在从根本上改变软件开发流程, 但优秀的工程师需要理解 LLM 的工作原理和局限性, 掌握 prompting 技术体系, 才能在 human-agent 协作中发挥最大价值。

4.1 关键点

- LLM 是基于 Transformer 的 next-token prediction 模型, 经过预训练、SFT 和偏好对齐三段训练
- Prompting 技术包括 zero-shot、k-shot、CoT、self-consistency、tool use、RAG 和 reflexion
- 好的 context engineering 是获得好结果的前提
- AI 不会取代开发者, 但会取代不会使用 AI 的开发者

4.2 拓展阅读

- 课程官网: <https://themodernsoftware.dev>
- 作业仓库: <https://github.com/mihail911/modern-software-dev-assignments>
- Anthropic Prompt Engineering Guide: <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/prompt-improver>
- Vaswani et al., “Attention Is All You Need” (2017)